

AWS BEDROCK INTEGRATION

Approach : IAM Role Assumption

AWS Non-Production Account

Scope	Environments	Auth Strategy	Cost
On-premises .NET 8 Web API → AWS Bedrock	Dev · QA · UAT	IAM Role Assumption via AWS STS	\$0 for IAM + STS

Executive Summary

This document outlines a secure, scalable, and cost-efficient approach for integrating AWS Bedrock with an on-premises .NET 8 Web API using IAM Role Assumption through AWS STS.

Key Benefits

- Uses temporary AWS credentials instead of long-lived Bedrock access
- Enforces least-privilege IAM security principles
- Automatically refreshes credentials using AWS SDK
- Isolates environments (Dev / QA / UAT) using dedicated IAM users
- Provides auditability through AWS CloudTrail
- Adds zero additional cost for IAM and STS usage

Recommended Approach: IAM Role Assumption provides the best balance of security, operational simplicity, maintainability, and cost optimization for non-production and shared environments.

1. Introduction

This document provides a step-by-step guide to integrating AWS Bedrock with an on-premises .NET 8 Web API using IAM Role Assumption as the authentication strategy. This is Approach 2 in a series of three credential management strategies evaluated for calling AWS Bedrock from an on-premises environment.

1.1 Credential Approaches — High Level Overview

Before diving into implementation, the table below summarises all three approaches and where this guide fits in:

#	Approach	How It Works	Credential Type	Best Suited For
1	IAM User Secret Keys	Long-lived Access Key + Secret stored directly in config. App calls Bedrock directly.	Permanent — never expire	Quick POC / local dev only
2	IAM Role Assumption ← This Guide	IAM User holds only sts:AssumeRole. AWS STS issues temporary credentials scoped to a role with Bedrock access.	Temporary — auto-expire (15 min – 12 hr)	Non-prod / Production / on-prem shared environments
3	IAM Roles Anywhere	No IAM User or long-lived key. Server presents X.509 certificate. AWS exchanges it for temporary credentials.	Temporary — cert-signed	Production / zero long-lived key tolerance

! Why Not Approach 1 for Shared Environments?

Approach 1 uses permanent Access Keys that never expire. If leaked, an attacker gains indefinite Bedrock access with no automatic remediation.

In shared non-prod environments with multiple team members or CI pipelines, this is a meaningful and avoidable risk.

Approach 2 eliminates permanent keys on the Bedrock call path — the credentials that actually invoke Bedrock always expire automatically.

1.2 Scope of This Guide

- Single AWS non-production account covering Dev, QA, and UAT environments
- One shared IAM Role: bedrock-role-nonprod
- On-premises .NET 8 Web API calling AWS Bedrock
- Least privilege IAM policies — only actions strictly required
- Bedrock Converse API — model-agnostic invocation overview
- Full AWS cost breakdown for all services involved
- CloudTrail audit trail details for the full authentication flow

2. How IAM Role Assumption Works

2.1 High-Level Concept

The key design shift from Approach 1 is the separation of identity (IAM User) from permissions (IAM Role):

Responsibility	Approach 1 — IAM User Keys	Approach 2 — Role Assumption
Who holds identity?	IAM User	IAM User (same)
Who holds Bedrock permissions?	IAM User (directly)	IAM Role (separate)
What credentials call Bedrock?	Permanent Access Key + Secret	Temporary token from STS (expires)
What can the IAM User do directly?	Everything the policy allows	Only: sts:AssumeRole

2.2 Authentication Flow

End-to-End Authentication Flow

```

Your .NET 8 API (On-Premises Server)
|
| [1] BasicAWSCredentials (IAM User Access Key + Secret Key)
|   Single permission only: sts:AssumeRole
|
v
AWS STS (Security Token Service)
|
| [2] AssumeRole Request --> bedrock-role-nonprod
|
| [3] Returns Temporary Credentials (duration: configurable)
|   AccessKeyId  -+
|   SecretKey    -+ All expire together at session end
|   SessionToken -+
|
v
AWS SDK (AssumeRoleAWSCredentials)
|
| [4] SDK auto-refreshes ~5 min before expiry
|   No manual timer or background service required
|
v
AWS Bedrock Runtime --> Claude / Haiku / Sonnet / Titan / Llama
  
```

2.3 Why This is Safer Than Approach 1

- The IAM User's Access Key cannot call Bedrock directly — it lacks the permission
- The only damage a leaked IAM User key can do is attempt an AssumeRole, which is fully logged in CloudTrail
- The temporary credentials that actually call Bedrock auto-expire — no manual rotation needed
- Each environment uses a separate IAM User, so a key compromise is isolated to one environment

3. AWS Console Setup

All setup is performed once in the non-production AWS account. The same role (bedrock-role-nonprod) is assumed by all three environments.

3.1 Step 1 — Create the IAM Role

Role Name	Account	Environments
bedrock-role-nonprod	Non-Prod AWS Account	Dev, QA, and UAT

In AWS Console: IAM → Roles → Create Role

- Trusted entity type → select AWS Account
- Select This account
- Click Next — do not attach policies yet
- Name the role: bedrock-role-nonprod
- Click Create Role

3.2 Step 2 — Attach Least Privilege Bedrock Policy to the Role

The role should hold only the two Bedrock actions required. Both the Converse API and InvokeModel API map to the same IAM actions — no separate Bedrock:Converse action exists.

Bedrock SDK Method Called	IAM Action Required
ConverseAsync (Converse API — used in this guide)	bedrock:InvokeModel
ConverseStreamAsync (Streaming Converse API)	bedrock:InvokeModelWithResponseStream
InvokeModelAsync (Direct invoke)	bedrock:InvokeModel
InvokeModelWithResponseStreamAsync	bedrock:InvokeModelWithResponseStream

Go to IAM → Roles → bedrock-role-nonprod → Permissions → Add permissions → Create inline policy → JSON tab:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "BedrockInvokeAllowedModels",
      "Effect": "Allow",
      "Action": [
        "bedrock:InvokeModel",
        "bedrock:InvokeModelWithResponseStream"
      ],
      "Resource": [
        "arn:aws:bedrock:us-east-1::foundation-model/anthropic.claude-haiku-4-5-20251001",
        "arn:aws:bedrock:us-east-1::foundation-model/anthropic.claude-3-5-haiku-20241022-v1:0",
        "arn:aws:bedrock:us-east-1::foundation-model/anthropic.claude-sonnet-4-20250514-v1:0"
      ]
    }
  ]
}
```

i Bedrock Model ARN Format

Model ARNs use a double colon (::) with no account ID — e.g. `arn:aws:bedrock:us-east-1::foundation-model/...`

Foundation models are AWS-managed, not account-owned.

The ARN must include the exact model ID string including version suffix.

Models must also be enabled in Bedrock → Model Access in your account before they can be invoked.

Actions Excluded from This Policy

Excluded Action	Why Not Needed
<code>bedrock:CreateModelCustomizationJob</code>	Fine-tuning — not used by this API
<code>bedrock:ListFoundationModels</code>	Model discovery — not required at runtime
<code>bedrock:GetFoundationModel</code>	Model metadata — not required at runtime
<code>bedrock:CreateProvisionedModelThroughput</code>	Reserved capacity — not applicable
<code>bedrock:PutModelInvocationLoggingConfiguration</code>	Logging admin — not a runtime need

3.3 Step 3 — Create IAM Users (One Per Environment)

Create three IAM users in the non-prod account. Each user has zero Bedrock permissions — only the right to assume `bedrock-role-nonprod`.

IAM User	Environment	Permission Granted
<code>bedrock-api-user-dev</code>	Development	<code>sts:AssumeRole</code> → <code>bedrock-role-nonprod</code> only
<code>bedrock-api-user-qa</code>	QA	<code>sts:AssumeRole</code> → <code>bedrock-role-nonprod</code> only
<code>bedrock-api-user-uat</code>	UAT	<code>sts:AssumeRole</code> → <code>bedrock-role-nonprod</code> only

For each user: IAM → Users → Create User → Permissions → Create inline policy:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowAssumeBedrockNonProdRole",
      "Effect": "Allow",
      "Action": "sts:AssumeRole",
      "Resource": "arn:aws:iam::YOUR_ACCOUNT_ID:role/bedrock-role-nonprod"
    }
  ]
}
```

3.4 Step 4 — Update the Role Trust Policy

The role must explicitly declare which IAM principals are permitted to assume it. Go to IAM → Roles → bedrock-role-nonprod → Trust relationships → Edit trust policy:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowNonProdUsersToAssume",
      "Effect": "Allow",
      "Principal": {
        "AWS": [
          "arn:aws:iam::YOUR_ACCOUNT_ID:user/bedrock-api-user-dev",
          "arn:aws:iam::YOUR_ACCOUNT_ID:user/bedrock-api-user-qa",
          "arn:aws:iam::YOUR_ACCOUNT_ID:user/bedrock-api-user-uat"
        ]
      },
      "Action": "sts:AssumeRole"
    }
  ]
}
```

3.5 Step 5 — Generate Access Keys

For each IAM user: IAM → Users → [username] → Security credentials → Create access key → Application running outside AWS. Save both values — the Secret Key is shown only once.

IAM User	Config File It Goes Into
bedrock-api-user-dev	appsettings.Development.json
bedrock-api-user-qa	appsettings.QA.json
bedrock-api-user-uat	appsettings.UAT.json

4. .NET 8 Integration Overview

This section provides a high-level overview of how the role assumption pattern is wired into the .NET 8 application. Full implementation code is covered in a separate technical specification.

4.1 NuGet Packages Required

Package	Purpose
AWSSDK.BedrockRuntime	Provides <code>IAmazonBedrockRuntime</code> — the Bedrock client
AWSSDK.SecurityToken	Provides <code>AssumeRoleAWSCredentials</code> and STS client

4.2 Configuration — Environment-Based Credential Loading

The application uses .NET's built-in layered configuration. A single environment variable (`ASPNETCORE_ENVIRONMENT`) on each server controls which credentials and model are loaded at startup — no code changes required between environments.

ASPNETCORE_ENVIRONMENT	Config File Loaded	Server
Development	<code>appsettings.Development.json</code>	Dev on-premises server
QA	<code>appsettings.QA.json</code>	QA on-premises server
UAT	<code>appsettings.UAT.json</code>	UAT on-premises server

! Security Reminder

Environment-specific config files should be placed in `PSConfig`.

4.3 Bedrock Client Registration — Role Assumption Logic

The following summarises the three-step pattern used in `Program.cs` to wire up the Bedrock client using assumed role credentials:

```
// Program.cs — Registration Overview

// Step 1: IAM User credentials — only permission is sts:AssumeRole
//     The user itself cannot call Bedrock directly
var userCredentials = new BasicAWSCredentials(accessKey, secretKey);

// Step 2: AssumeRoleAWSCredentials — SDK handles temp credential refresh
//     automatically renews ~5 minutes before expiry. No manual refresh needed.
var assumeRoleCredentials = new AssumeRoleAWSCredentials(
    userCredentials, // used only to call STS, not Bedrock
    roleArn,         // arn:aws:iam::ACCOUNT:role/bedrock-role-nonprod
    sessionName      // e.g. bedrock-session-dev — appears in CloudTrail
);

// Step 3: Bedrock client receives temporary role credentials, not user keys
//     Credentials auto-refresh for the lifetime of the application
```

```
builder.Services.AddSingleton<IAmazonBedrockRuntime>(_ =>
    new AmazonBedrockRuntimeClient(assumeRoleCredentials, region));
```

i AssumeRoleAWSCredentials — Automatic Refresh

The AWS SDK calls STS internally on first use and automatically refreshes temporary credentials approximately 5 minutes before expiry.

No background refresh service, retry logic, or scheduled task is required.

This makes it safe for APIs that run continuously across environment restarts.

4.4 Bedrock Converse API — Why Model-Agnostic

The application uses the Bedrock Converse API rather than InvokeModel. The key benefit is a single unified request/response shape that works across all foundation models — switching models requires only a config change.

Aspect	InvokeModel API	Converse API (Used Here)
Request body format	Different JSON schema per model vendor	Single unified schema for all models
Response parsing	Different per model	Always: response.Output.Message.Content[0].Text
Switching models	Requires code changes	Change ModelId config value only
IAM action required	bedrock:InvokeModel	bedrock:InvokeModel (identical)
Streaming support	InvokeModelWithResponseStream	ConverseStreamAsync (same shape)

Environment	Model	Rationale
Dev / QA / UAT	Claude 3.5 Haiku	Lowest cost — suitable for developer testing and iteration
If required, can be updated	Claude Haiku 4.5	Faster, higher capability — suitable based on requirement

5. CloudTrail Audit Trail

5.1 What CloudTrail Captures

AWS CloudTrail records all API activity in your account. Approach 2 produces a richer audit trail than Approach 1 because authentication involves two distinct, separately logged events — the role assumption and the model invocation.

Event	CloudTrail Event Name	Source	What Is Logged
IAM User assumes the role	AssumeRole	sts.amazonaws.com	IAM user ARN, role ARN, session name, source IP, timestamp, success/failure
Temp credentials call Bedrock	InvokeModel	bedrock.amazonaws.com	Role session ARN, model ID, region, source IP, timestamp, HTTP status

Note: CloudTrail management events are captured by default in Event History — no configuration needed.

5.2 AssumeRole Event — What to Look For

Every time the SDK refreshes temporary credentials, an AssumeRole event is logged. This is a management event and is recorded in CloudTrail at no additional cost.

```
// Sample AssumeRole CloudTrail Entry (simplified)
{
  "eventName": "AssumeRole",
  "eventSource": "sts.amazonaws.com",
  "userIdentity": {
    "type": "IAMUser",
    "arn": "arn:aws:iam::YOUR_ACCOUNT:user/bedrock-api-user-dev"
  },
  "requestParameters": {
    "roleArn": "arn:aws:iam::YOUR_ACCOUNT:role/bedrock-role-nonprod",
    "roleSessionName": "bedrock-session-dev"
  },
  "sourceIPAddress": "203.0.113.45",
  "eventTime": "2025-09-01T08:00:00Z"
}
```

5.3 Bedrock InvokeModel Event — What to Look For

Each Bedrock call is also logged. The `userIdentity` reflects the assumed role session, not the original IAM user — this identifies which environment made the call.

```
// Sample Bedrock InvokeModel CloudTrail Entry (simplified)
{
  "eventName": "InvokeModel",
  "eventSource": "bedrock.amazonaws.com",
  "userIdentity": {
    "type": "AssumedRole",
    "arn": "arn:aws:sts::YOUR_ACCOUNT:assumed-role/bedrock-role-nonprod/bedrock-session-dev"
  },
  "requestParameters": {
```

```
"modelId": "anthropic.claude-3-5-haiku-20241022-v1:0"
},
"sourceIPAddress": "203.0.113.45",
"eventTime": "2025-09-01T08:01:05Z",
"responseElements": null
}
```

5.4 Viewing Events in AWS Console

- Navigate to AWS Console → CloudTrail → Event history
- Filter by Event name: AssumeRole to see all credential refresh events
- Filter by Event name: InvokeModel to see all Bedrock calls
- Filter by Username: bedrock-api-user-dev (or qa / uat) to isolate per environment
- Events are retained in Event history for 90 days at no additional cost

5.5 CloudTrail Cost

Event Type	Cost
Management events (AssumeRole, IAM changes)	Free — first copy always free
Data events (Bedrock InvokeModel calls)	\$0.10 per 100,000 events if data event logging is explicitly enabled
CloudTrail Lake (long-term query storage)	Separate pricing — optional, not required

i Data Events Are Optional

Bedrock InvokeModel calls are data events and are NOT logged by default. Management events (AssumeRole) are always logged for free and are sufficient for most non-production audit requirements.

NOTE: Bedrock InvokeModel data event logging (a paid service) is not being enabled in CloudTrail for either non-prod or production.

6. AWS Cost Breakdown

6.1 Services Cost Summary

AWS Service	What Is Used	Cost
IAM	Users, Roles, Policies	Always free
STS	AssumeRole API calls for temp credentials	Always free
CloudTrail	AssumeRole audit logs (management events)	Free — first copy always free
CloudTrail (optional)	Bedrock InvokeModel data event logging	\$0.10 per 100,000 events if enabled
Bedrock Runtime	Converse / InvokeModel API — per token	Paid — see below

6.2 Bedrock Pricing — Model Comparison

Bedrock charges per 1,000 input tokens and per 1,000 output tokens. Approximately 1,000 tokens equals 750 words of text.

Model	Input / 1K Tokens	Output / 1K Tokens	Relative Cost	Suggested Use
Claude 3.5 Haiku	\$0.0008	\$0.004	Lowest	Non-prod — developer testing, iteration
Claude Haiku 4.5	\$0.0010	\$0.005	Low	Higher envs if required — faster capability

6.3 Estimated Monthly Cost Example

i Assumptions

250 API calls per day per environment
 Average 300 input tokens + 400 output tokens per call
 22 working days per month

Dev Environment — Claude 3.5 Haiku
 Input: $250 \times 300 / 1000 \times \$0.0008 \times 22 = \$1.32$ / month
 Output: $250 \times 400 / 1000 \times \$0.0040 \times 22 = \$8.80$ / month
 Total Dev: ~\$10.12 / month

Dev Environment — Claude Haiku 4.5 (if upgraded)
 Input: $250 \times 300 / 1000 \times \$0.0010 \times 22 = \$1.65$ / month
 Output: $250 \times 400 / 1000 \times \$0.0050 \times 22 = \$11.00$ / month
 Total: ~\$12.65 / month

IAM + STS: \$0.00 (always free)

Total Estimated DEV Cost: ~\$10 / month at this volume

i Cost Takeaway

The Role Assumption mechanism — IAM and STS — adds zero cost versus Approach 1.
 Your entire AWS bill is driven by Bedrock token usage.
 Selecting Claude 3.5 Haiku for Dev / QA / UAT environments keeps non-prod costs minimal.

7. Summary

7.1 What Was Set Up

Component	Detail
IAM Role	bedrock-role-nonprod — shared across Dev/QA/UAT; holds all Bedrock permissions
IAM Users	3 environment-specific users — each holds only sts:AssumeRole; zero direct Bedrock access
Trust Policy	Role explicitly trusts all three IAM users to assume it
Credential type on Bedrock path	Temporary — STS-issued, 1-hour default, auto-refreshed by SDK
Bedrock API	Converse API — model-agnostic, unified request and response shape
Model selection	Config-driven per environment — zero code change to switch models
Cost of auth mechanism	\$0.00 — IAM and STS are always free
Audit trail	AssumeRole + InvokeModel events logged in CloudTrail per environment

→ Approach 3: IAM Roles Anywhere

Approach 3 eliminates IAM Users and long-lived Access Keys entirely. The on-premises server presents an X.509 certificate to a Roles Anywhere trust anchor in AWS, which exchanges it for temporary credentials scoped to a role. This removes the last remaining long-lived secret and is the recommended approach for production environments with zero long-lived key tolerance.

Operational Tradeoff

Approach	Security	Cost	Complexity
IAM User Access Keys	Low	Free	Simple
STS AssumeRole with base key ← Recommended	Medium	Free	Medium
IAM Roles Anywhere + existing CA	High	Medium	Medium
IAM Roles Anywhere + AWS Private CA	Very High	Expensive	Managed

i Recommendation

Best Balance — Security + Cost + Simplicity: IAM Role Assumption using AWS STS temporary credentials.